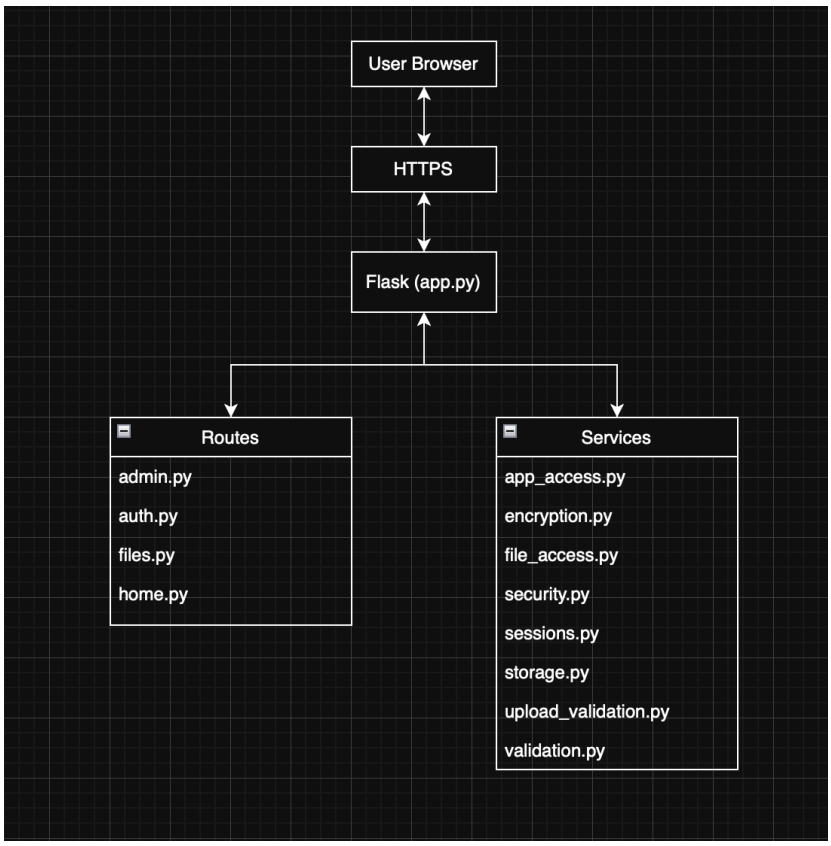
Security Design Document

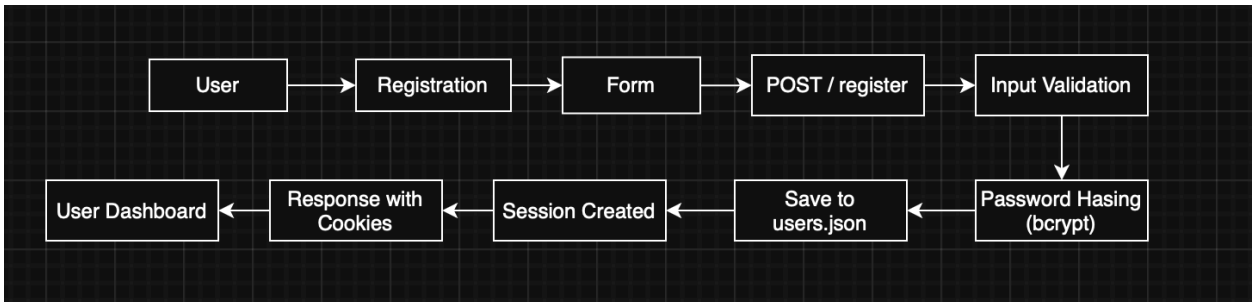
A. Architecture Overview (10 points)

1. System Architecture Diagram

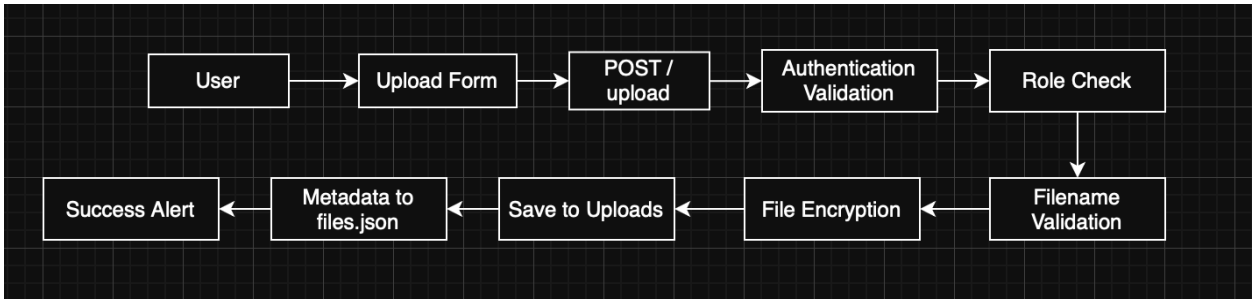


2. Data Flow Diagrams

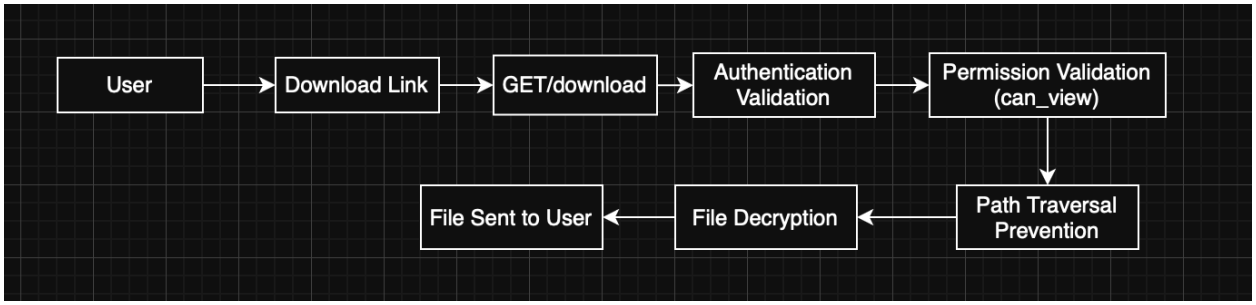
Example 1: User Registration Flow



Example 2: File Upload Flow



Example 3: File Download Flow



3. Component Descriptions

Component	Description
Flask Application	Main web application framework, that handles everything from HTTP requests, different routings, and response generations.
Auth Routes	This is where we handle user registration, login, logout, password validation, `bcrypt` hashing, account lockout, and rate limiting. Our general file for everything authentication.
File Routes	This file handles all of our file related operations like file upload, download, modifying, deleting and sharing. All files are encrypted and linked with access control for proper users.
Admin Routes	Our administrative functions live here which include user management, account locking, session termination, password resets, and admin logs.
Home Routes	This is our dashboard display, which includes file listings, and public file access.
Security Service	Contains all critical security functions which includes password validation, email validation, all event loggings, and a temporary password generator.
File Access Service	This file handles our file permission checking, such as who can view, edit, delete or modify certain files.

Validation Service	Provides input sanitization, filename validation, path traversal prevention, XSS control, and general length checking.
Session Manager	Creates, validates, and destroys our server side sessions with 30-minute timeouts and ensures a secure cookie attribute.
Storage Service	This file handles the reading and writing of our JSON files for users, files, shares, and security logs.
Upload Validation	This file handles extension allowlist and content/MIME sniffing + blocking detected potential malware.
Encryption Service	This key file manages our encryption keys, and provides file encryption and decryption for our stored documents.

4. Technology Stack Justification

Technology	**Reasoning**
Python	Heavily familiarized; easy to read code; strong many trusted libraries.
JSON File Storage	No large database setup required, good readability which makes for easy debugging. We believed JSON was large enough for our project scope.
Flask	We have used Flask before so we are fairly familiar with it, it's easy to learn and has built in security features. It's also widely documented and not a heavy item to run considering project requirements.
`Jinja2`	Comes with Flask; auto-escapes HTML output in `.html` templates, which defends against XSS attacks.
`bcrypt`	One of the strongest industry password hashing's which has built in salting, and resists brute force attacks.
`Fernet`	Part of `cryptography` library, which is well-known and provides secure authenticated symmetric encryption. Using this library was easy and eliminated the risk of badly implementing our own encryption algorithm.
`Email-Validator`	It is an easy-to-work with email validator that takes the complex regex involved with validating emails out of our hands, which eliminates the risk of improper regexes that could lead to bad emails being allowed. Also verifies the existence of MX records through DNS.
`Filetype`	Manual implementation of MIME sniffing becomes easy to mess up with the large variety of file types out there. Using a dedicated library significantly lessens the risk of misidentifying files.

`Werkzeug`	This is a well-established library, and it handles sanitizing filenames for us. By using this instead of implementing it ourselves, we lessen the risk of incorrectly sanitizing, potentially allowing path traversal attacks.

B. Threat Model (10 points)

1. Asset Identification

Asset	**How its Protected**	**Sensitivity**
Encryption Keys	Persisted to disk in `data/` and exposed internally only to `EncryptedStorage` and `FileEncryptor` objects	Critical (Critical)
Session Tokens	Stored in `Fernet`-encrypted `sessions.json`; transmitted via cookie (`HttpOnly,Secure,SameSite=Strict`)	Critical (Critical)
User Password (hash)	`bcrypt` hashes stored in `Fernet`-encrypted `users.json`	Critical (Critical)
User PII/Account Metadata	Stored in `Fernet`-encrypted `users.json`	High (Confidential)
Uploaded Documents	`Fernet`-encrypted (`uploads.key`)	High (Confidential)
Security Logs	Stored in `Fernet`-encrypted `security.log`	High (Confidential)
File Metadata	Stored in `Fernet`-encrypted `files.json`	Medium (Internal)
Public Documents	`Fernet`-encrypted (`uploads.key`)	Low (Public)

2. Threat Enumeration

Aspect	**Description**
Brute Force	An attacker can use automated tools to continuously try different password combinations against a known username until the correct password is found
Unauthorized File Access	A user may attempt to access, download, edit or delete files they do not own/ files they don't have permission too. This includes horizontal/vertical escalation
Malicious File Upload	An attacker can upload a file containing malicious content which is meant to compromise the server and execute code.

Session Hijacking	An attacker can steal a valid user's session token and use it in order to impersonate the victim. This would allow them to gain access to their account without needing their password
Data Leakage	Sensitive information may be exposed to unauthorized parties through either error messages, HTTPS response headers, logs or something else
Bad Actor / Internal Threat	Admin can gain access to user accounts by resetting their password and using that password to login to their account.

3. Vulnerability Assessment (see Penetration Report)

Vulnerability	**Severity**	**Justification**
Password reset is performed manually by admin	High	Temporary password exposed directly to admin; admin can login as user.
Rate limiting can be bypassed via proxy	Medium	The per-IP counter uses `request.remote_addr`; can be bypassed with VPN/proxy rotation.
Session hijacking	Medium	Session tokens are hard to steal with HTTPS and secure cookies. But if they are stolen, it could be bad.
Sever header reveals version info	Low	This only gives the attacker information on what potential attacks they can use depending on the libraries we used.
Race conditions in concurrent sessions	Low	Last write wins. But user can just login again to establish a new session.

4. Attack Scenarios

Scenario 1: Brute Force Password Attack

Goal: Guess a user's password
Method: Try an automated login system with common passwords
Mitigation: Rate Limiting will only allow 10 attempts per minute, and after 5 failed attempts the account gets locked.

Scenario 2: Session Hijacking

Goal: Impersonate a logged in user
Method: Steal the users active session token through network sniffing or XSS
Mitigation: Secure cookies, HTTPS only, timeouts

Scenario 3: Privilege Escalation

Goal: Gain admin powers as a regular user
Method: Direct URL access to /admin/user through the URL
Mitigation: RBAC

Scenario 4: Malicious File Upload

Goal: Execute code on the server via file
Method: Upload file with script name
Mitigation: Filename validation, content encryption

5. Risk Prioritization

Brute Force Attack - Medium Priority (rate limit + lockout, but proxy bypass)
Session Hijacking - Medium Priority (token should be hard to get, but no IP binding)
Privilege Escalation - Low Priority (already have RBAC + `@require_role` verified)
Data breach via file - LowPriority (uploads are checked)

C. Security Controls (10 points)

i. User Authentication - Registration

1. Control Description

The registration controls make sure that only real users with valid credentials can create an account. It validates user inputs from the username to the password, enforces password complexity, checks for duplicate account names, and stores all passwords using `bcrypt`. All attempts to register are also logged for security.

2. Implementation Approach

All our inputs were sanitized, our username was validated with 3-20 chars, and alphanumeric characters. We check email format and also validate passwords by ensuring they have 12+ chars, upper and lowercase letters, numbers, and special characters. We also check for duplicate usernames in `user.json`, while our password is hashed with `bcrypt`.

3. Testing Methodology

A valid registration shows successful, invalid username shows error, duplicate username is rejected, weak passwords are rejected, any missing fields are alerted and all XSS attempts are blocked.

4. Known Limitations

Email validation is very basic, and there is no rate limiting on registration.

5. Mitigation Strategies

Add an email verification similar to sending a code to the registered email to ensure its validity. Add rate limiting from the same IP when registering.

ii. User Authentication - Login

1. Control Description

The login authenticates users against stored `bcrypt` hashes. It implements rate limiting for logging in, and includes account lockout. When successful it creates a secure session.

2. Implementation Approach

Rate limiting tracks attempts per IP, username is sanitized and the user is looked up in `user.json`. On success, we reset the failed attempts, create a session and attach a secure cookie. On failure, we increment a counter before lockout. All events are logged.

3. Testing Methodology

Valid login, bring you to the main page. Invalid password shows error, 5 wrong attempts logs you out. 10 attempts in one minute triggers the rate limit. Logs show success, failures and lockouts.

4. Known Limitations

Rate limiting is based on `remote_addr`, and can be bypassed via proxy. The dictionary `login_attempts` is stored in-memory, which resets on every application restart.

5. Mitigation Strategies

Add an additional IP check and ensure a proxy can not bypass the rate limiting. Keep track of `login_attempts` in persistent storage.

iii. Access Control - RBAC

1. Control Description

Restricts access based on roles, admins have full access, user is the standard with most general permissions, and guest the final read only role.

2. Implementation Approach

Implemented using a role hierarchy where admin is the highest at 3. Created permission functions such as `can_view()`, `can_edit()`, `can_delete()`, and `can_share()` to delegate who can do what. We also use `get_file_role_for_user()`, to determine a user's role on a specific file.

3. Testing Methodology

A regular user trying to access the admin panel through URL is met with a `403 Forbidden Error`. User A cannot do anything to User B's file unless given permission. Guests cannot access and protect pages.

4. Known Limitations

Roles may be too broad; admin has too much power.

5. Mitigation Strategies

Add additional roles to take responsibilities away from admin.

iv. Input Validation - XSS Prevention

1. Control Description

Prevent malicious script execution, while also protecting against stored XSS.

2. Implementation Approach

We implemented `validation.py` which includes sanitizing inputs and outputs, to ensure that fields like username could not be injected with any script.

3. Testing Methodology

Attempted registration with `<script>alert('XSS')</script>`, where nothing occurs.

4. Known Limitations

Current CSP headers, specifically `'unsafe-inline'` for `script-src` and `style-src`, introduces an injection point that bypasses `html.escape` through inline JavaScript.

5. Mitigation Strategies

Add explicit validation to reject JavaScript inline scripts.

v. Input Validation - Command Injection Prevention

1. Control Description

Prevents execution of commands through a user's filename.

2. Implementation Approach

Implemented `safe_filename()` which validates the filename based on metacharacters, regex and extension.

3. Testing Methodology

Entering a `.pdf` file extension is rejected with invalid filename. Valid file names like `test1.txt` are accepted with no errors. Logs show any validation failures.

4. Known Limitations

Reject a legitimate filename because it has a space, or character in another language. No length limit.

5. Mitigation Strategies

Add unicode support, add a max filename length like 255 characters.

vi. Input Validation - Path Traversal Prevention

1. Control Description

Prevents users from accessing files outside the directory through a change in URL.

2. Implementation Approach

Also implemented in `validation.py`, it uses the same `secure_filename()` method to remove any directory paths. Raises a ValueError when a traversal is detected.

3. Testing Methodology

Entering in addition the URL with `/download/filename` returns an invalid screen. Valid downloads pass, while suspicious ones are logged.

4. Known Limitations

`Werkseug.secure_filename()` strips non-ASCII characters, so filenames with unicode characters are messed up without raising errors.

5. Mitigation Strategies

Validate the filename before passing into `secure_filename()`; explicitly raise errors on filenames containing non-ASCII characters instead of silently letting them through.

vii. Encryption - Transport Encryption

1. Control Description

Encrypts all clients to server communications using TLS. This prevents browser sniffing and man in the middle attacks.

2. Implementation Approach

Implemented in `app.py`, we create `cert.pem` and `key.pem`. Creates self signed certificate with OpenSSL for 365 days, redirects HTTP to a not found screen. Secure cookies.

3. Testing Methodology

HTTP requests will be redirected to HTTPS if the app is reached over HTTP. But in the current setup, no HTTP listener is bound, so HTTP attempts lead to a not found screen instead. Headers were verified using browser development tools.

4. Known Limitations

Certificate currently self generated.

5. Mitigation Strategies

Work to create a trusted certificate, and set up automatic renewal.

viii. Encryption - Data at Rest

1. Control Description

Encrypts all uploaded files to protect content from unauthorized filesystem access.

2. Implementation Approach

Two-key separation between `secret.key` and `uploads.key`. File metadata, sessions, user info/metadata, file-shares data, and security log uses `secret.key`. Documents use `uploads.key`.

3. Testing Methodology

Upload file is unreadable, but downloaded file is readable. `secret.key` does not exist on GitHub.

4. Known Limitations

Single key for all files, our key is also stored on the same server as files. No key rotation.

5. Mitigation Strategies

Use a different key per file, which is encrypted with a master key. Store the master key away in a different server and implement a key rotation every 90 days.

ix. Session Management - Secure Session Implementation

1. Control Description

Handles secure session creations, destruction, and validation with tokens, timeouts and secure cookies.

2. Implementation Approach

Tokens are generated with proper methods, while session metadata is stored in `sessions.json`. An extra function like `validate_session()` checks if a token exists already and the last time it was used.

3. Testing Methodology

Cookie shows secure flags, a new token is created on login. Cookie is cleared out on logout. A new cookie is given after every login.

4. Known Limitations

No IP binding. Race condition as JSON files are stored without locking.

5. Mitigation Strategies

Add IP binding, and implement a file lock for the JSON files.

x. Security Headers

1. Control Description

Add HTTP response headers that create browser security features like XSS protection and sessionjacking prevention.

2. Implementation Approach

Added to `app.py` where headers are set to a list of pre-determined values to combat against XSS attacks, sniffing and more.

3. Testing Methodology

Headers were verified browser development tools, viewing past the certificate, to see every response header.

4. Known Limitations

Shows versions for certain systems like python, which can allow an attacker to abuse known limitations for that version.

5. Mitigation Strategies

Remove server header.

xi. Security Logging

1. Control Description

Record and report all security relevant events from authentication, authorization, data access, lockouts, session changes, suspicious activity, and validation failures.

2. Implementation Approach

Log entries include an event type, username, IP address, details, user-agent, and timestamp.

3. Testing Methodology

Each event type that was triggered is shown in the log file. For example failed logins produce a `LOGIN_FAILED` entry that can be viewed by any admin through the admin panel. You can test any of these, like `ACCOUNT_LOCKOUT` or more.

4. Known Limitations

No real time alerts.

5. Mitigation Strategies

Provide notifications to admins currently logged in, or to developers when there is an important logged event.

D. Data Protection (10 points)

1. Data Classification

Tier / Label	Description	Examples
Tier 1 — Critical	Exposure gives an attacker access to accounts and encrypted data.	Authentication credentials, session tokens, `Fernet` encryption keys.
Tier 2 — Confidential	Exposure gives an attacker access to data that may require a certain role.	User personal information, file contents, security logs.
Tier 3 — Internal	Exposure gives an attacker information about data.	File metadata (original filenames, stored filenames, owner, share records).
Tier 4 — Public	Exposure does not give an attacker meaningful information.	Files explicitly marked as public by owner.

2. Encryption Methods

`Fernet` uses AES-128-CBC and HMAC-SHA256 for authenticated symmetric encryption. This ensures confidentiality and integrity.

Data at Rest - Structured Storage

Structured data stores (users, files metadata, shares between users, sessions, and security logs) are encrypted using `Fernet` in `encryption.py` via the `EncryptedStorage` class. The key, `secret.key`, is generated and stored in the `EncryptedStorage` object with no way to access it directly. The key is only used by the object itself to encrypt and decrypt metadata.

Data at Rest - Uploaded Files

Uploaded files are encrypted using `Fernet` in `encryption.py` via the `FileEncryptor` class. The key, `uploads.key`, is generated and stored in the `FileEncryptor` object with no way to access it directly. The key is only used by the object itself to encrypt and decrypt file contents.

Data in Transit

The application requires HTTPS. An automatic redirect from HTTP to HTTPS is performed before any request to ensure no traffic is exposed through HTTP. Then, transport-layer security headers are set:

`X-Frame-Options`: `DENY` → prevents clickjacking via iframe embedding

`X-Content-Type-Options`: `nosniff` → prevents MIME sniffing attacks

`X-XSS-Protection`: `1; mode=block` → XSS protection for legacy browsers

`Referrer-Policy`: `strict-origin-when-cross-origin` → limits referrer leakage

`Strict-Transport-Security`: `max-age=31536000; includeSubDomains` → enforces HSTS across all subdomains for one year

`Permissions-Policy`: `('geolocation=(), microphone=(), camera=()')` → denies listed permissions

Password Hashing

User passwords are never stored in plaintext. Passwords are salted and hashed with a work factor of 12 to prevent brute force and rainbow table attacks using `bcrypt`. `bcrypt` is used in every instance that requires password storage: admin user creation, user registration, password reset.

3. Key Management

Key Generation

Keys are generated using `cryptography.fernet.Fernet.generate_key()`. This produces a cryptographically random URL-safe base64-encoded 32-byte key. There are two keys: `secret.key` and `uploads.key`.

Key Storage (and Bootstrapping)

Keys are stored on the disk in the `data/` directory of the project on first startup. If a key file does not exist, it will be generated by the necessary class.

Key Separation

Structured data and file content use different keys to ensure that compromise of one key does not directly expose the data in the other group.

Key Rotation

Currently no automatic key rotation. Manual key rotation requires stopping the app, generating a new key, decrypting all existing data with the old key and re-encrypt with the new key, then replacing the key on disk.

4. Secure Deletion Procedures

File Deletion

When an authorized user deletes a file:

1. The file is removed from disk via `os.remove()`.
2. The file's metadata record in `data/files.json` is removed.
3. All records in `data/shares.json` that reference the deleted file are removed.

This ensures that the file is thoroughly deleted and nothing can reference it after it has been deleted.
It is worth mentioning that `os.remove()` does not cryptographically overwrite data, it only unlinks the file from the filesystem.

Session Deletion

Sessions are removed from the encrypted session storage in four situations:

1. Voluntary logout — session token is removed and session cookies are cleared.
2. Admin force-close — a specific session token is removed; the user trying to use that token will not be able to do anything with an expired token and will be redirected to the login page.
3. User demotion — all sessions associated with a certain user will be forced closed; same behavior as scenario 2.
4. Idle timeout — if a session token goes a certain amount of time without being updated (through any session activity); the timed out session will be automatically purged on next request (from anyone, not just session owner).